

.AIM / DESCRIPTION

The aim of this project is to

1. calculate the fill-level of a Caritas-Container used for collection of second hand clothes in Salzburg using a ultrasonic sensor
2. send the resulting values to a server over mobile-network and display them accordingly

in order to improve route- and time-planning in the Caritas logistics-center (especially when it comes to containers with a few hours of travel distance by car), save on fuel and have an even more positive impact on the environment.

Currently, the prototype, packed into a 3D printed case, is tested in one of their containers directly on site.

.BILL OF MATERIALS & PARTS

1x LILYGO TTGO T-Call ESP32 (2G network GPRS, SIM Card SIM800L Module)

(In future revisions there might be an upgrade to a 4G sim module depending on developments in the 2G network sector)

1x LifePo4 18650 3.7V 1500mAh rechargeable battery

1x 18650 battery holder

1x Liitokala Lii-PD4 battery charger

1x USB-C cable

1x HC-SR04 Sensor 5V

(a US-100 Ultrasonic Sensor (2.4-5.5V) or HC-SR04 wide voltage (3.3-5.5V) will be used in future revisions as power consumption can be optimized much better.)

4x Male to Female jumper cables

1x 3D-Printed case consisting of

- Board basecase
- Board backplate
- Sensor basecase
- Sensor backplate
- Battery holder base

2x M4 Screws (4x16mm)

2x M4 Wing nuts

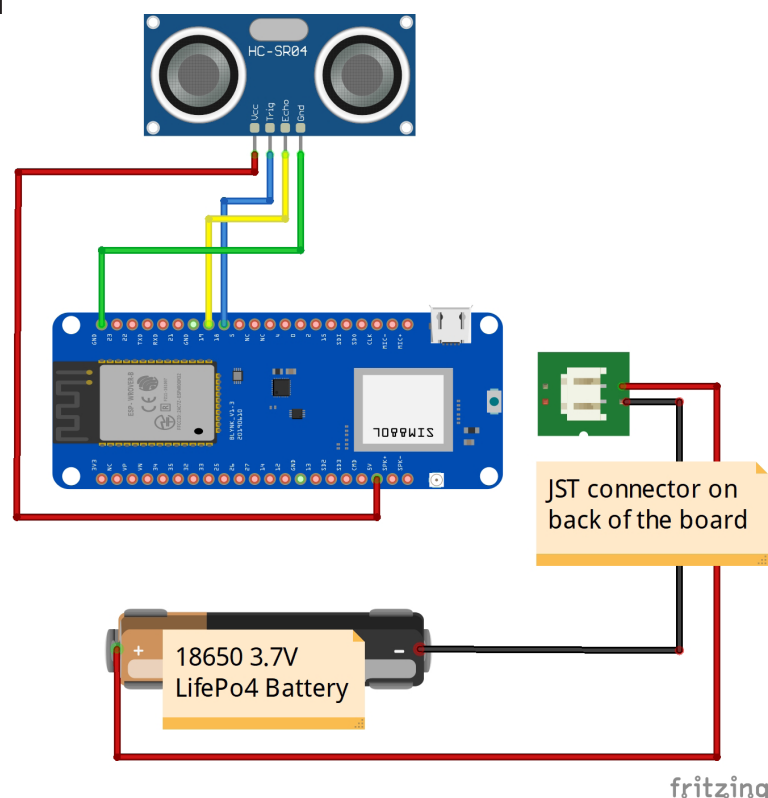
2x M4 flat washers

1x Nano sim-card and associated data-plan

.NECESSARY TOOLS

- Solder iron
- Solder
- A holding tool ("third hand") for soldering purposes is highly recommended
- Access to 3D Printer
- Access to a smartphone with Blynk app
- Blynk account
- Arduino compatible IDE
- Board Manager:
 - esp32* from espressif
- Libraries:
 - TinyGSM* from Volodymyr Shymanskyi
 - blynk-library* from blynkkk
 - Wire* from Arduino
 - QuickMedianLib* from Luis LLamas
 - esp_sleep* from espressif
 - WiFi* from espressif
 - esp_bt* from espressif
 - esp_wifi* from espressif
 - Wire* from Arduino
 - HCSR04* from Martin Susic
 - QuickMedianLib* from Luis LLamas

.CIRCUIT DIAGRAM



.SHORT DESCRIPTION OF SOFTWARE ROUTINE

.credentials

In the associated .ino (arduino) file, first and foremost the GPRS credentials for connecting to the phone-data-network and *Blynk* authentication are handled as to simplify changes for eventual third parties in the future. *Blynk* has been chosen for testing purposes only (and might be relaced with another tool), the auth code will change depending on static or dynamic provisioning.

The various GPRS credentials, depending on data-carrier, can be found here: <https://web.archive.org/web/20180119161650/http://wiki.apnchanger.org>

Next, the sim-pin is defined or left empty if disabled.

.pins

First, the *GPIO* pins for all internal systems are defined: *Modem* as well as *I2C* connections. Second, the *Trigger* and *Echo*-pins for the connected *HC-SR04 ultrasonic sensor*.

.deep sleep time

We define the time for the board to sleep between routines defined in micro seconds. A conversion factor is implemented so we can use seconds instead. The datatype should be *unsigned long long* as to avert under-or over-flows and incorrectly working sleep schedules.

.serial prints

Serial objects are defined and can be commented in or out to respectively enable or disable debug output.

SerialAT should never be disabled as it is necessary for communication between board and modem.

.GSM modem

Modem type, buffer size and adresses for *I2C* cinnections are defined.

.objects

TinyGSM modem, *I2C TwoWire* as well as *Ultrasonic-Distance sensor*-objects are created.

.variables

Variables used in the following software routine are defined such as connection states and mounting height of sensor.

.setup and loop

It should be noted that the typical setup and routine-sections will only be used as a superficial division of code as the board will go back to deep-sleep if it completes its routine or runs into problems but doesn't actually loop.

Thus, in setup we start with disabling all unnecessary peripherals like Wifi, Bluetooth, analog-digital converter and set the cpu frequency to lowest possible with *ESP32* (10Mhz).

We start I2C and modem communication, set various pin-modes and configure the timers wake up source.

The sim-card will now be unlocked if a pin has been set and the modem will try to connect to the mobile data-network as well as the *Blynk*-servers afterwards or send the board back to deepsleep if it fails.

In loop, first the battery percentage and voltage is measured with the integrated battery power manager ([AXP192](#)).

As those values seem to be less than reliable, a different approach will have to be found in future iterations.

Continuing, multiple measurements will be taken with the ultrasonic-distance sensor and fed into the [QuickMedianLib](#) library to prune out false readings and lower the impact of values affected by noise.

The median will be used to calculate the fill-level (in percent), very much depending on the mounting-height of the sensor.

After a check if the unit is still connected to the *Blynk*-servers, the fill-level and battery values are sent, afterwards sending the board back to a deep slumber.

.EXTERNAL RESOURCES

GPRS credentials

<https://web.archive.org/web/20180119161650/http://wiki.apnchanger.org>

RandomnerdTutorials

<https://randomnerdtutorials.com/esp32-sim800l-publish-data-to-cloud/>

Tips to increase battery life of ESP32

<https://savjee.be/2019/12/esp32-tips-to-increase-battery-life/>

Reducing energy consumption of ESP32

<https://diyi0t.com/reduce-the-esp32-power-consumption/>

Choosing a battery for ESP32

<https://diyi0t.com/best-battery-for-esp32/>

LilyGo Github with datasheets, Schematics, example code

<https://github.com/Xinyuan-LilyGO/LilyGo-T-Call-SIM800>

Espressif

<https://github.com/espressif>

Volodymyr Shymanskyy

<https://github.com/vshymanskyy>

Blynk

<https://github.com/blynkkk>

Arduino

<https://github.com/arduino>

Luis Llamas

<https://github.com/luisllamasbinaburo>

Martin Sosic

<https://github.com/Martinsos>

.EXTERNAL RESOURCES

GPRS credentials

<https://web.archive.org/web/20180119161650/http://wiki.apnchanger.org>

RandomnerdTutorials

<https://randomnerdtutorials.com/esp32-sim800l-publish-data-to-cloud/>

Tips to increase battery life of ESP32

<https://savjee.be/2019/12/esp32-tips-to-increase-battery-life/>

Reducing energy consumption of ESP32

<https://diyi0t.com/reduce-the-esp32-power-consumption/>

Choosing a battery for ESP32

<https://diyi0t.com/best-battery-for-esp32/>

LilyGo Github with datasheets, Schematics, example code

<https://github.com/Xinyuan-LilyGO/LilyGo-T-Call-SIM800>

Espressif

<https://github.com/espressif>

Volodymyr Shymanskyy

<https://github.com/vshymanskyy>

Blynk

<https://github.com/blynkkk>

Arduino

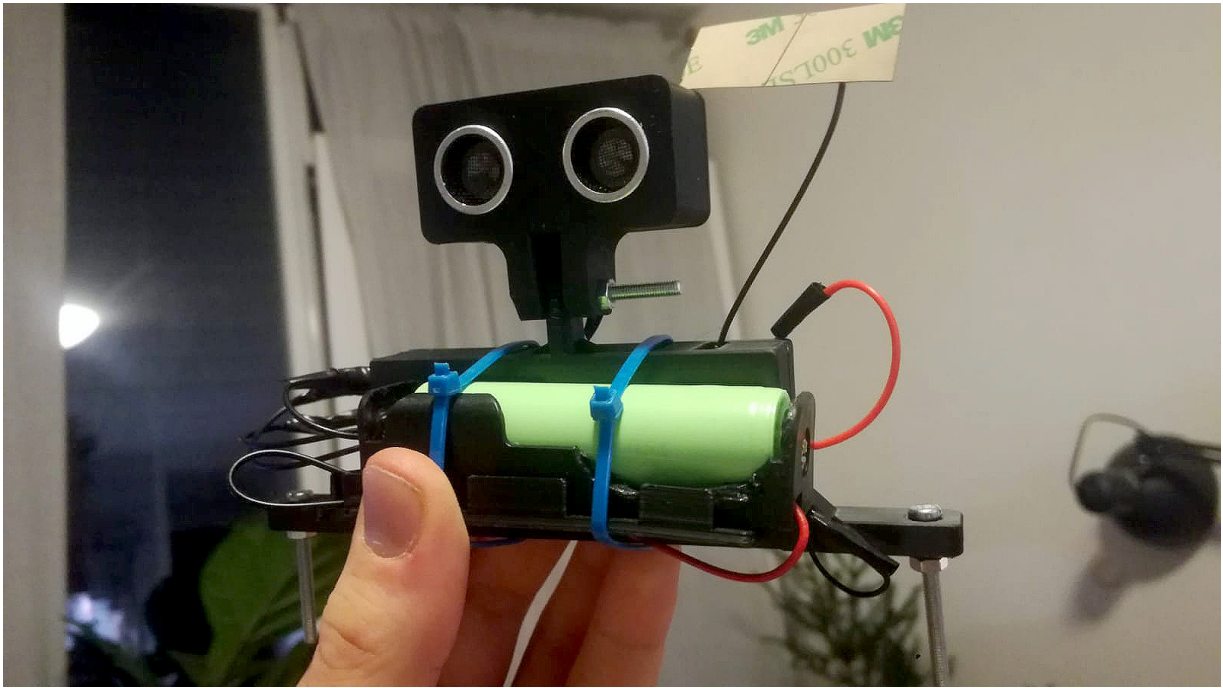
<https://github.com/arduino>

Luis Llamas

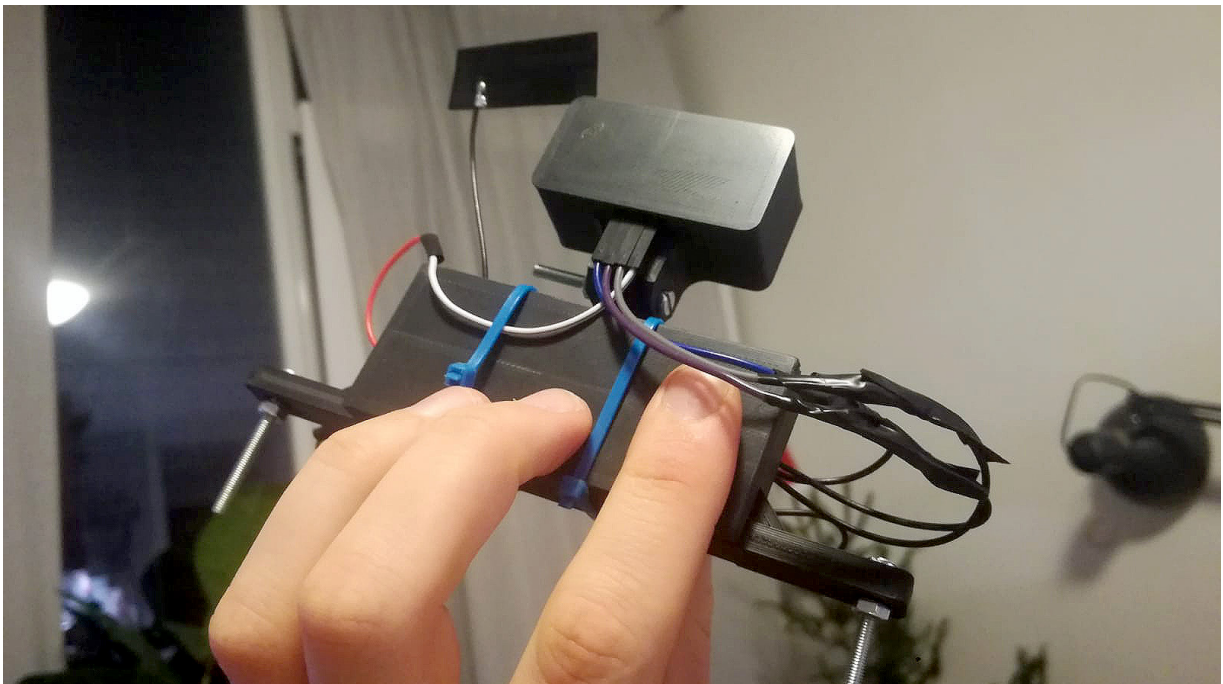
<https://github.com/luisllamasbinaburo>

Martin Sosic

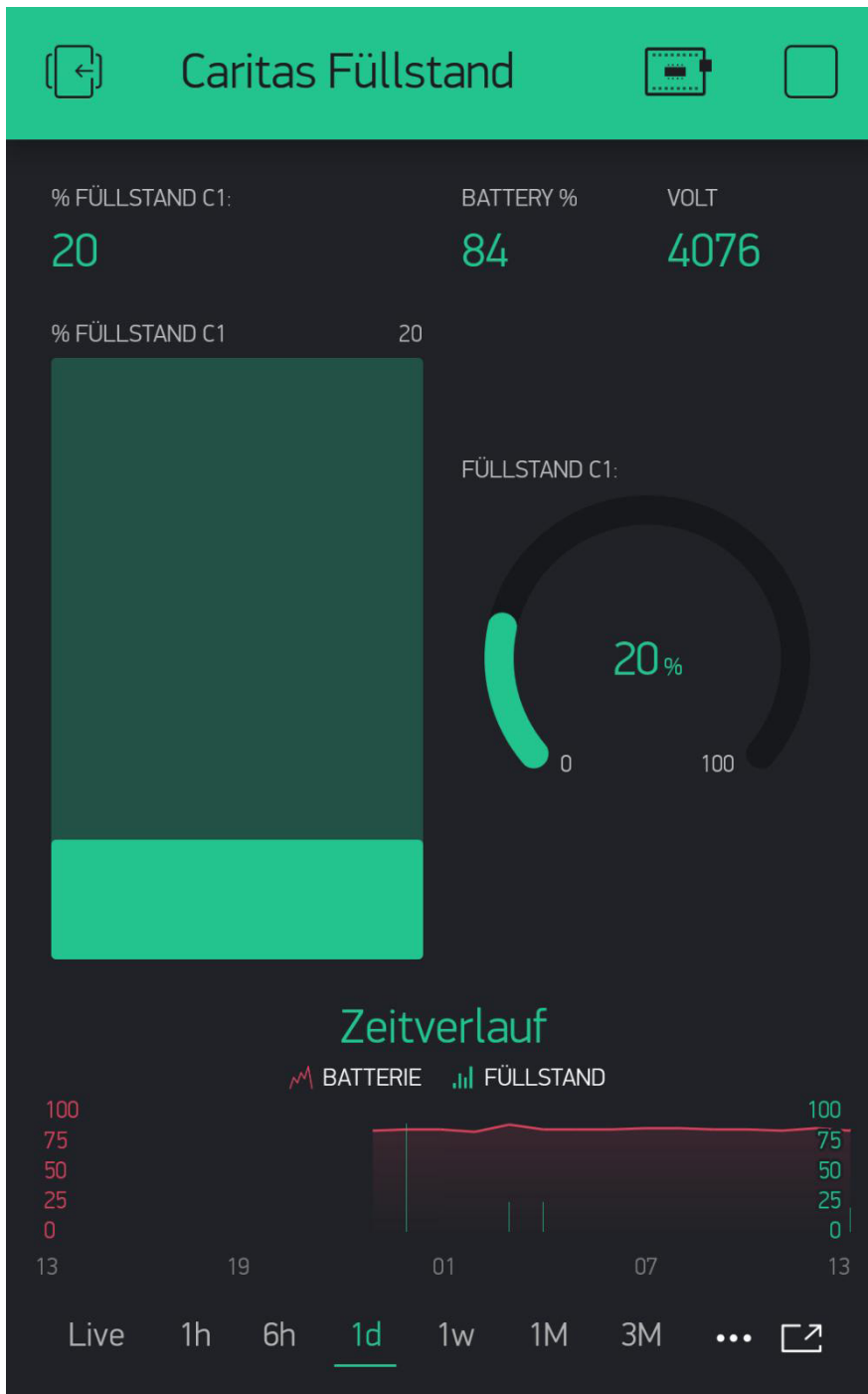
<https://github.com/Martinsos>



Assembled unit, front



Assembled unit, back



Values displayed in Blynk